Contents

ParkOpticon YOLO11s - Project Onboarding Guide **1**
1. Project Overview . . . 2
    1.1 Goal . . . 2
    1.2 Key Innovation . . . 2
2. Detection Classes . . . 2
    Why 4 Classes? . . . 2
3. Data Sources . . . 2
    3.1 Street View Backgrounds . . . 2
    3.2 Reference Images for Synthesis . . . 3
    3.3 Synthetic Training Data . . . 3
4. The Pipeline . . . 3
    4.1 Pipeline Overview . . . 3
    4.2 Step-by-Step . . . 4
5. Synthetic Data Generation Deep Dive . . . 5
    5.1 How It Works . . . 5
    5.2 Prompt Engineering . . . 5
    5.3 Image Differencing for Labels . . . 6
    5.4 Current State . . . 6
6. Training Configuration . . . 6
    6.1 Default Settings . . . 6
    6.2 Dataset YAML . . . 6
    6.3 Output Artifacts . . . 7
7. Important Constraints & Gotchas . . . 7
    7.1 Data Leakage Prevention . . . 7
    7.2 Sibling Management . . . 7
    7.3 Empty Frame Detection Threshold . . . 7
    7.4 Consistent Preprocessing . . . 7
8. File Structure . . . 7
9. Quick Commands Reference . . . 8
10. API Keys Required . . . 9
11. Troubleshooting . . . 9
    "No empty candidates found" . . . 9
    "Synthesis failed" . . . 9
    "Out of memory during training" . . . 9
    "Labels don't match vehicles" . . . 10
12. Next Steps for Current Project . . . 10

ParkOpticon YOLO11s - Project Onboarding Guide

Last Updated: March 3, 2026

---

1. Project Overview

1.1 Goal

Train a YOLO11s object detection model to identify vehicles in street-level imagery, with special focus on detecting parking enforcement vehicles and police cruisers. The model will be used to automate parking violation detection in Ottawa, Ontario.

1.2 Key Innovation

This project uses **100% synthetic training data** generated via Google's Gemini API. Instead of manually collecting and labeling thousands of images, we:

1. Collect empty street scene backgrounds from Google Street View
2. Use generative AI to insert vehicles into these scenes
3. Automatically generate bounding box labels using image differencing

This approach cost only **$6.23** in API credits to generate 2,658 synthetic training images.

---

2. Detection Classes

The model detects **4 classes**:

Class ID	Name	Description
0	`vehicle`	Regular vehicles (cars, trucks, buses, motorcycles)
1	`enforcement_vehicle`	Parking enforcement vehicles (Bylaw vehicles)
2	`police_old`	Ottawa Police cruisers with old livery
3	`police_new`	Ottawa Police cruisers with new livery (facelift design)

Why 4 Classes?

- **Vehicle (Class 0):** The base class for all regular traffic. Provides negative examples for enforcement classes.
- **Enforcement Vehicle (Class 1):** Parking enforcement vehicles are distinct from regular police and have unique livery.
- **Police Old/New (Classes 2, 3):** Ottawa Police is transitioning to a new livery design. Both variants exist on roads and need separate detection.

---

3. Data Sources

3.1 Street View Backgrounds

Source: Google Street View Static API

Location: `data/images_original/`

Structure:

```
data/images_original/
+-- auto_{pano_id}_h0/
|   +-- {hash}.jpg
+-- auto_{pano_id}_h45/
|   +-- {hash}.jpg
+-- ...
```

Each panorama is captured at 8 heading angles (0°, 45°, 90°, 135°, 180°, 225°, 270°, 315°).

Statistics: - Total images: 9,600 - Unique panoramas: 1,200 - Headings per panorama: 8

3.2 Reference Images for Synthesis

These directories contain real photos of enforcement vehicles used as style references for Gemini:

Directory	Purpose	Count
`enforcement-dataset/`	Bylaw/parking enforcement vehicles	~18 images
`police-dataset-old/`	Ottawa Police old livery cruisers	~18 images
`police-dataset-new/`	Ottawa Police new livery cruisers	~17 images

Important: These are **reference images only**, not training data. They guide the AI during synthesis.

3.3 Synthetic Training Data

Location: `data/images_synth/`

Structure:

```
data/images_synth/
-- random_vehicle/      # Class 0: Regular vehicles (1,451 images)
-- enforcement_vehicle/ # Class 1: Bylaw vehicles (475 images)
-- police_old/          # Class 2: Old livery cruisers (467 images)
-- police_new/          # Class 3: New livery cruisers (464 images)
```

Total: 2,857 synthetic images (after generation, before manual review)

---

4. The Pipeline

4.1 Pipeline Overview

```
+---------------+    +---------------+    +---------------+
|  1. FETCH     |--->|  2. CROP      |--->|  3. EMPTY     |
|  Street View  |    |  Remove bot.  |    |  Detect empty |
```

```
|  Images       |    |  30px overlay |    |  frames       |
+---------------+    +---------------+    +---------------+
                                                    |
                                                    v
+---------------+    +---------------+    +---------------+
|  6. SPLIT     |<---|  5. LABELER   |<---|  4. SYNTH     |
|  Train/Val/Te |    |  Manual review|    |  Generate AI  |
|  Group-aware  |    |  of labels    |    |  images       |
+---------------+    +---------------+    +---------------+
         |
         v
+---------------+    +---------------+
|  7. TRAIN     |--->|  8. EVAL      |
|  YOLO11s      |    |  Test set     |
|  Fine-tune    |    |  metrics      |
+---------------+    +---------------+
```

4.2 Step-by-Step

Step 1: Fetch Images (`make fetch`)

- Downloads Street View images using the points manifest
- Creates `manifests/images.csv` with metadata
- **Idempotent:** Use `--resume` to continue interrupted downloads

Step 2: Crop Images (`make crop`)

- Removes bottom 30 pixels (copyright watermarks, UI overlays)
- Creates sibling files with `_bc30` suffix
- Updates manifest to reference cropped versions
- **Idempotent:** Skips already-cropped images

Step 3: Detect Empty Frames (`make empty`)

- Uses pretrained YOLO11s (COCO weights) to detect vehicles
- Confidence threshold: 0.15-0.25 (low to avoid false negatives)
- Outputs: `lists/empty_candidates.txt`
- **Critical:** Lower threshold prevents missing vehicles that would create bad synthetic labels

Step 4: Synthesize (`make synth`)

- Uses Google Gemini API (gemini-3-pro-image-preview)
- For each empty frame, generates:
  - 1 random_vehicle (no reference needed)
  - 1 enforcement_vehicle (with reference image)
  - 1 police_old (with reference image)
  - 1 police_new (with reference image)
- **Cost:** ~$6.23 for 2,658 images
- **Time:** ~1 hour 11 minutes

Step 5: Prelabel (`make prelabel`)

- For synthetic images: Uses image differencing to find inserted vehicle
- For original images: Uses pretrained YOLO to detect vehicles
- Outputs YOLO format labels to `data/labels_autogen/`

Step 6: Manual Review (`make labeler`)

- Launches web UI at `http://localhost:8000`
- Review and correct auto-generated labels
- Delete nonsensical synthetic images
- Save final labels to `data/labels_final/`

Step 7: Split Dataset (`make split`)

- Group-aware splitting by `pano_id`
- Prevents data leakage (same location in train/test)
- Default ratios: 70% train, 20% val, 10% test
- Creates `data/splits/{train,val,test}/`

Step 8: Train (`make train`)

- Fine-tunes YOLO11s on custom dataset
- Default: 50 epochs, 640x640 images
- Outputs: `runs/detect/parkopticon_vehicle_enforcement/weights/`

Step 9: Evaluate (`make eval`)

- Runs inference on test set
- Generates metrics report

---

5. Synthetic Data Generation Deep Dive

5.1 How It Works

The synthesis script (`scripts/04_synthesize_vehicle_edits.py`) uses Gemini's image editing capabilities:

1. **Input:** Empty street scene + (optionally) reference image
2. **Prompt:** Detailed instructions to insert a vehicle matching the reference style
3. **Output:** Edited image with vehicle inserted

5.2 Prompt Engineering

The prompts include strict constraints:

```
- Scene Lock: Preserve original geometry, don't move camera or buildings
- Local Edit Only: Modify only pixels near the inserted vehicle
- Traffic Direction: Vehicle must face correct way for lane
- Shadow Consistency: Match scene lighting (hard shadows for sunny, soft for overcast)
- Size Constraint: Output must match input dimensions exactly
```

5.3 Image Differencing for Labels

After synthesis, we detect where the vehicle was inserted:

1. Load original empty frame and synthetic frame
2. Compute absolute difference
3. Threshold and morphological close
4. Find largest contour → bounding box
5. Convert to YOLO format (normalized center + dimensions)

5.4 Current State

Class	Images Generated	Status
random_vehicle	1,451	Needs review
enforcement_vehicle	475	Needs review
police_old	467	Needs review
police_new	464	Needs review
Total	**2,857**	**Pending manual review**

Next Step: Manual review to delete nonsensical images and verify labels.

---

6. Training Configuration

6.1 Default Settings

Parameter	Value	Notes
Base Model	YOLO11s	Pretrained on COCO
Epochs	50	Adjustable via `--epochs`
Image Size	640x640	Standard for YOLO
Batch Size	Auto	Determined by available memory
Device	CPU	Use `--device 0` for GPU

6.2 Dataset YAML

Located at `dataset.yaml`:

```yaml
path: data/splits
train: train/images
val: val/images
test: test/images

names:
  0: vehicle
  1: enforcement_vehicle
  2: police_old
  3: police_new
```

```yaml
nc: 4
```

6.3 Output Artifacts

After training: - `runs/detect/parkopticon_vehicle_enforcement/weights/best.pt` - Best checkpoint - `runs/detect/parkopticon_vehicle_enforcement/weights/last.pt` - Final checkpoint - `runs/detect/parkopticon_vehicle_enforcement/results.png` - Training curves - `reports/train_summary.md` - Metrics summary

---

7. Important Constraints & Gotchas

7.1 Data Leakage Prevention

Critical: Always split by `pano_id`, never by image.

Each panorama has 8 heading angles showing the same intersection from different directions. If these end up in different splits, the model memorizes the location instead of learning to detect vehicles.

The split script handles this automatically by grouping on `pano_id`.

7.2 Sibling Management

Synthetic images are "siblings" of their parent empty frame. They must stay in the same split:

```
Parent: auto_abc123_h0.jpg (empty frame)
  +-- auto_abc123_h0_random_vehicle.jpg
  +-- auto_abc123_h0_enforcement_vehicle.jpg
  +-- auto_abc123_h0_police_old.jpg
  +-- auto_abc123_h0_police_new.jpg
```

All5 images must be in train OR val OR test—never mixed.

7.3 Empty Frame Detection Threshold

Use **low confidence (0.15-0.25)** for empty frame detection.

If you use a higher threshold, you might miss a partially occluded vehicle. Then you'd insert a synthetic vehicle into a frame that already has one, creating a confusing label.

7.4 Consistent Preprocessing

If you crop images during training, you **must** apply the same crop at inference time. Mismatched preprocessing degrades model performance.

---

8. File Structure

```
parkopticon-yolo11s-bootstrap/
+-- scripts/                    # Pipeline scripts (numbered by order)
```

```
|   +-- 01_make_points_template.py
|   +-- 01b_expand_points_by_area.py
|   +-- 02_fetch_streetview.py
|   +-- 02b_crop_bottom.py
|   +-- 03_detect_empty_frames.py
|   +-- 04_synthesize_vehicle_edits.py
|   +-- 05_prelabel_boxes.py
|   +-- 06_split_dataset.py
|   +-- 07_train_yolo.py
|   +-- 08_eval_report.py
|
+-- labeler/                    # Web-based labeling UI
|   +-- app.py                  # FastAPI backend
|   +-- static/
|       +-- index.html
|       +-- app.js
|
+-- data/
|   +-- images_original/        # Street View images (cropped)
|   +-- images_synth/           # Synthetic generated images
|   |   +-- random_vehicle/
|   |   +-- enforcement_vehicle/
|   |   +-- police_old/
|   |   +-- police_new/
|   +-- labels_autogen/         # Auto-generated labels
|   +-- labels_final/           # Manually reviewed labels
|   +-- splits/                 # Train/val/test splits
|
+-- manifests/
|   +-- points.csv              # Location manifest (input)
|   +-- images.csv              # Image manifest (generated)
|
+-- enforcement-dataset/        # Reference: Bylaw vehicles
+-- police-dataset-old/         # Reference: Old livery cruisers
+-- police-dataset-new/         # Reference: New livery cruisers
|
+-- dataset.yaml                # YOLO dataset config
+-- Makefile                    # Pipeline commands
+-- requirements.txt            # Python dependencies
+-- .env                        # API keys (not in git)
```

---

9. Quick Commands Reference

```python
# Setup
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate
```

```
pip install -r requirements.txt
cp .env.example .env      # Then add your API keys

# Run pipeline
make fetch      # Download Street View images
make crop       # Remove bottom overlay
make empty      # Find empty frames
make synth      # Generate synthetic images
make prelabel   # Auto-generate labels
make labeler    # Launch labeling UI
make split      # Create train/val/test split
make train      # Train YOLO11s
make eval       # Evaluate on test set

# Help
make help
```

---

10. API Keys Required

Key	Source	Purpose
`GSV_API_KEY`	Google Cloud Console	Street View Static API
`GEMINI_API_KEY`	Google AI Studio	Synthetic image generation

Add these to `.env`:

```
GSV_API_KEY=your_key_here
GEMINI_API_KEY=your_key_here
```

---

11. Troubleshooting

"No empty candidates found"

- Try lower confidence threshold: `python scripts/03_detect_empty_frames.py --conf 0.15`
- Check if images actually contain vehicles (visual inspection)

"Synthesis failed"

- Check Gemini API quota
- Verify `GEMINI_API_KEY` in `.env`
- Try different model: `--model gemini-1.5-pro`

"Out of memory during training"

- Reduce batch size: `python scripts/07_train_yolo.py --batch 8`
- Use smaller image size: `--imgsz 416`

"Labels don't match vehicles"

- Image differencing can fail on subtle insertions
- Use the labeler UI to manually correct

---

12. Next Steps for Current Project

1. **Manual Review** - Go through `data/images_synth/` and delete nonsensical images
2. **Label Verification** - Use `make labeler` to verify/correct bounding boxes
3. **Dataset Split** - Run `make split` after review
4. **Training** - Run `make train` with GPU if available
5. **Evaluation** - Analyze per-class performance, especially for rare classes

---

Document generated for ParkOpticon YOLO11s Bootstrap project. Author: Esad Kaya | University of Ottawa | SEG 4180